

Harsha Kyle
192311087

STUDENT RECORD GUI

PROGRAM OUTPUT SCREENSHOT

Student Record Management

STUDENT RECORD MANAGEMENT SYSTEM

Student Details

Roll Number:

Name:

Branch:

GPA:

Roll No	Name	Branch	GPA
192311087	Harsha Kyle	CSE	9.2
192311088	Rahul Sharma	IT	8.9

SOURCE CODE

```
import javax.swing.*;
import javax.swing.border.EmptyBorder;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.util.ArrayList;
import java.util.List;

public class StudentRecordGUI extends JFrame {
    private final JTextField rollField = new JTextField();
    private final JTextField nameField = new JTextField();
    private final JTextField branchField = new JTextField();
    private final JTextField gpaField = new JTextField();

    private final DefaultTableModel tableModel;
    private final JTable studentTable;
    private final List<Student> studentsList = new ArrayList<>();

    public StudentRecordGUI() {
        setTitle("Student Record Management");
        setSize(700, 450);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);

        JPanel mainPanel = new JPanel(new BorderLayout(12, 12));
        mainPanel.setBorder(new EmptyBorder(15, 15, 15, 15));

        JLabel titleLabel = new JLabel("STUDENT RECORD MANAGEMENT SYSTEM",
SwingConstants.CENTER);
        titleLabel.setFont(new Font("SansSerif", Font.BOLD, 20));
        mainPanel.add(titleLabel, BorderLayout.NORTH);

        JPanel formPanel = new JPanel(new GridLayout(5, 2, 8, 8));
        formPanel.setBorder(BorderFactory.createTitledBorder("Student Details"));
        formPanel.setPreferredSize(new Dimension(280, 200));

        formPanel.add(new JLabel("Roll Number:"));
        formPanel.add(rollField);
        formPanel.add(new JLabel("Name:"));
        formPanel.add(nameField);
        formPanel.add(new JLabel("Branch:"));
        formPanel.add(branchField);
        formPanel.add(new JLabel("GPA:"));
        formPanel.add(gpaField);

        JButton addBtn = new JButton("Add");
        JButton updateBtn = new JButton("Update");
        JButton deleteBtn = new JButton("Delete");
        JButton clearBtn = new JButton("Clear");

        JPanel buttonPanel = new JPanel(new GridLayout(2, 2, 6, 6));
        buttonPanel.add(addBtn);
        buttonPanel.add(updateBtn);
        buttonPanel.add(deleteBtn);
        buttonPanel.add(clearBtn);

        JPanel leftPanel = new JPanel(new BorderLayout(8, 8));
        leftPanel.add(formPanel, BorderLayout.CENTER);
        leftPanel.add(buttonPanel, BorderLayout.SOUTH);
        mainPanel.add(leftPanel, BorderLayout.WEST);

        tableModel = new DefaultTableModel(new String[]{"Roll No", "Name", "Branch",
```

```

"GPA"}, 0) {
    @Override
    public boolean isCellEditable(int row, int col) { return false; }
};
studentTable = new JTable(tableModel);
studentTable.setSelectionMode(ListSelectionModel.SINGLE_SELECTION);
JScrollPane tableScroll = new JScrollPane(studentTable);
mainPanel.add(tableScroll, BorderLayout.CENTER);

addBtn.addActionListener(e -> addStudent());
updateBtn.addActionListener(e -> updateStudent());
deleteBtn.addActionListener(e -> deleteStudent());
clearBtn.addActionListener(e -> clearFields());

studentTable.getSelectionModel().addListSelectionListener(e -> {
    int selectedRow = studentTable.getSelectedRow();
    if (selectedRow >= 0) {
        rollField.setText(tableModel.getValueAt(selectedRow, 0).toString());
        nameField.setText(tableModel.getValueAt(selectedRow, 1).toString());
        branchField.setText(tableModel.getValueAt(selectedRow, 2).toString());
        gpaField.setText(tableModel.getValueAt(selectedRow, 3).toString());
        rollField.setEditable(false);
    }
});

add(mainPanel);
setVisible(true);

studentsList.add(new Student("192311087", "Harsha Kyle", "CSE", "9.2"));
studentsList.add(new Student("192311088", "Rahul Sharma", "IT", "8.9"));
refreshTable();
}

private void addStudent() {
    if (!validateInput()) return;
    String roll = rollField.getText().trim();
    for (Student s : studentsList) {
        if (s.roll.equals(roll)) {
            JOptionPane.showMessageDialog(this, "Student Roll No already exists.",
"Error", JOptionPane.ERROR_MESSAGE);
            return;
        }
    }
    studentsList.add(new Student(roll, nameField.getText().trim(),
branchField.getText().trim(), gpaField.getText().trim()));
    refreshTable();
    JOptionPane.showMessageDialog(this, "Student added successfully.", "Success",
JOptionPane.INFORMATION_MESSAGE);
    clearFields();
}

private void updateStudent() {
    int selectedRow = studentTable.getSelectedRow();
    if (selectedRow < 0) {
        JOptionPane.showMessageDialog(this, "Select a record from table to update.",
"Warning", JOptionPane.WARNING_MESSAGE);
        return;
    }
    if (!validateInput()) return;
    String roll = rollField.getText().trim();
    for (Student s : studentsList) {
        if (s.roll.equals(roll)) {
            s.name = nameField.getText().trim();
            s.branch = branchField.getText().trim();

```

```

        s.gpa = gpaField.getText().trim();
        break;
    }
}
refreshTable();
JOptionPane.showMessageDialog(this, "Student updated successfully.", "Success",
JOptionPane.INFORMATION_MESSAGE);
clearFields();
}

private void deleteStudent() {
    int selectedRow = studentTable.getSelectedRow();
    if (selectedRow < 0) {
        JOptionPane.showMessageDialog(this, "Select a record from table to delete.",
"Warning", JOptionPane.WARNING_MESSAGE);
        return;
    }
    String roll = tableModel.getValueAt(selectedRow, 0).toString();
    int choice = JOptionPane.showConfirmDialog(this, "Are you sure you want to delete
this student record?", "Confirm Deletion", JOptionPane.YES_NO_OPTION);
    if (choice == JOptionPane.YES_OPTION) {
        studentsList.removeIf(s -> s.roll.equals(roll));
        refreshTable();
        JOptionPane.showMessageDialog(this, "Student deleted successfully.", "Success",
JOptionPane.INFORMATION_MESSAGE);
        clearFields();
    }
}

private void refreshTable() {
    tableModel.setRowCount(0);
    for (Student s : studentsList) {
        tableModel.addRow(new Object[]{s.roll, s.name, s.branch, s.gpa});
    }
}

private void clearFields() {
    rollField.setText("");
    nameField.setText("");
    branchField.setText("");
    gpaField.setText("");
    rollField.setEditable(true);
    studentTable.clearSelection();
}

private boolean validateInput() {
    if (rollField.getText().trim().isEmpty() ||
        nameField.getText().trim().isEmpty() ||
        branchField.getText().trim().isEmpty() ||
        gpaField.getText().trim().isEmpty()) {
        JOptionPane.showMessageDialog(this, "All student fields are required.",
"Validation Error", JOptionPane.ERROR_MESSAGE);
        return false;
    }
    try {
        double val = Double.parseDouble(gpaField.getText().trim());
        if (val < 0.0 || val > 10.0) {
            JOptionPane.showMessageDialog(this, "GPA must be between 0.0 and 10.0",
"Validation Error", JOptionPane.ERROR_MESSAGE);
            return false;
        }
    } catch (NumberFormatException e) {
        JOptionPane.showMessageDialog(this, "GPA must be a valid float value.",
"Validation Error", JOptionPane.ERROR_MESSAGE);
    }
}

```

```
        return false;
    }
    return true;
}

static class Student {
    String roll, name, branch, gpa;
    Student(String roll, String name, String branch, String gpa) {
        this.roll = roll;
        this.name = name;
        this.branch = branch;
        this.gpa = gpa;
    }
}

public static void main(String[] args) {
    SwingUtilities.invokeLater(StudentRecordGUI::new);
}
}
```